Way2meal App – Requirements & Architecture (Phase 1)

Nov 5, 2026 · @vijinbk

Overview and scope

Phase 1 replaces WhatsApp order-taking with an Android app for customers and a web admin panel for the kitchen, with payments recorded manually and OTP stubbed. Razorpay, a real SMS OTP provider and the Google Play listing come in Phase 2, behind interfaces built now so they plug in without rework.

Business today: Way2meal is a cloud kitchen selling high-protein diet meals (35 g+ protein, 350–450 kcal each). There is a fixed menu per weekday for Lunch and Dinner, Monday to Saturday. Customers buy a 1-day trial, a weekly plan (6 meals) or a monthly plan (26 meals). Orders must be placed before 10 AM.

Goals for Phase 1

- Customers see the day's menu, subscribe, book single meals, add extras, and skip days themselves.
- Skipped subscription meals carry forward automatically, extending the plan end date.
- The kitchen gets a daily prep count and delivery list without reading WhatsApp chats.
- The admin edits menus, prices, add-ons, holidays and cutoff times without a developer.

In scope vs later

Area	Phase 1 (now)	Phase 2 (later)
Login	Phone number + OTP via a mock provider (fixed dev OTP, or OTP shown in admin panel)	Real SMS provider (e.g. MSG91, Twilio, Firebase Phone Auth)
Payments	Customer pays by UPI/cash outside the app; uploads UTR or screenshot; admin marks paid	Razorpay checkout, webhooks, auto-activation, refunds via API
Distribution	Signed APK shared directly / Expo internal build	Google Play listing; iOS optional
Notifications	In-app notices; push via Expo (free) optional	WhatsApp Business API / SMS reminders

Area	Phase 1 (now)	Phase 2 (later)
Delivery	Admin assigns delivery person; list view	Live rider tracking (optional)

Out of scope for now: multiple kitchens, rider app, coupons, loyalty points, nutrition tracking.

Users, roles and plan types

There are three roles and two kinds of customer: **subscribers** get the fixed menu automatically and can skip; **trial/daily customers** book a specific day's meal.

Role	Uses	Can do
Customer	Android app	Register, manage addresses, buy plans, book meals, add extras, skip, view history and payments
Admin (owner)	Web admin panel	Everything: menus, plans, prices, add-ons, customers, payments, holidays, settings, reports
Kitchen / delivery staff	Web admin panel (limited)	View today's prep sheet and delivery list, mark meals delivered

Plan types (each plan is for one meal slot: Lunch or Dinner; a customer may hold both)

Plan	Meals	Lunch price (₹)	Dinner price (₹)	Gets fixed menu automatically	Can skip / carry forward
1-day trial	1	169	169	No – books a chosen date	No
Daily (pay per meal)	1 per booking	admin-set	admin-set	No – books a chosen date	No (can cancel before cutoff)
Weekly	6	990	960	Yes, Mon–Sat	Yes
Monthly	26	4159	4160	Yes, Mon–Sat	Yes

All prices, meal counts and limits are admin-editable data, never hard-coded. A trial should be limited to once per phone number (admin-configurable).

Business rules

The core rule: a subscription is a set of meal credits scheduled onto service days; a skip moves one credit to the next free service day, so the plan simply ends later.

Service days and time

- All dates and cutoffs are in IST (Asia/Kolkata). Store timestamps in UTC, compute days in IST.
- Service days are Monday–Saturday. Sunday is off (admin-configurable weekday set).
- Admin-declared holidays are not service days; meals scheduled on a holiday move automatically to the end of the plan.

Cutoff

- Default cutoff is 10:00 AM on the delivery day for both Lunch and Dinner (configurable per slot, e.g. Dinner 2 PM).
- Before cutoff, a customer can book, cancel, skip, unskip or add extras for that day. After cutoff the day is locked for the customer; only the admin can change it.
- Booking for future dates is allowed up to N days ahead (default 7).

Subscription scheduling

1. When a weekly/monthly plan is activated (payment confirmed), the system creates one `Delivery` row per meal on the next N service days from the chosen start date.
2. Start date: earliest is tomorrow, or today if before cutoff.
3. `endDate` = date of the last scheduled delivery.

Skip and carry forward

1. Customer skips a scheduled date before cutoff → that delivery becomes `SKIPPED` and a new `SCHEDULED` delivery is appended on the next service day after the current last delivery. Plan `endDate` moves out.
2. Unskip before cutoff → restore the original date and remove the most recently appended carry-forward row.
3. Bulk skip a date range (e.g. travelling) is supported; each date follows rule 1.
4. Limits (admin settings): max skips per plan (default: weekly 2, monthly 8) and carry-forward validity (default: plan must complete within 2× its normal length). When the limit is reached, the skip button explains why it is disabled.
5. Optional "pause plan": skip all dates from X until resumed, same mechanics.
6. Admin can grant extra carry-forward credits (e.g. kitchen missed a delivery) and can refund unused credits; refunds are recorded manually in Phase 1.

Trial and daily bookings

- Customer picks a date and slot, sees that date's menu, adds extras, places the order.
- Capacity: admin sets max meals per slot per day; booking closes when full or after cutoff.
- Cancel before cutoff → credit or refund recorded by admin.

Add-ons (extras)

- Admin-managed catalogue, e.g. Premium Protein Veg Salad (300 g: 100 g chicken breast + 200 g veg salad), extra 100 g chicken breast, extra fruit bowl, extra yogurt.
- Each add-on has price, slot availability (lunch/dinner/both), active flag, optional daily stock.
- Subscribers attach add-ons to any scheduled delivery before cutoff; trial/daily customers add them to their order. Add-ons are always paid separately (Phase 1: marked as payable and settled with admin).
- An add-on attached to a skipped day is cancelled, not carried forward.

Menu

- Menu = weekly template per weekday per slot (as in the posters), plus date-specific overrides the admin can set for a single date (e.g. festival special).
- The menu shown for any date = override if one exists, else the weekday template.
- Dinner offers 6 chicken flavours; a customer may set a preferred flavour per delivery or a default on their profile ("Kitchen's choice" if none).

Payments (Phase 1, manual)

- Order or plan is created as `PENDING_PAYMENT`. The app shows the kitchen's UPI ID/QR and amount.
- Customer enters the UPI reference (UTR) and optionally uploads a screenshot.
- Admin verifies and marks `PAID` → plan activates and deliveries are scheduled. Admin can also mark cash payments.
- All amounts stored as integer paise; INR only.

Delivery status

`SCHEDULED → SKIPPED | PREPARING → OUT_FOR_DELIVERY → DELIVERED | MISSED`. A `MISSED` delivery (kitchen's fault) can be re-credited by admin.

Functional requirements

The customer app has 5 tabs (Home, Menu, My Plan, Orders, Profile); the admin panel has 10 pages.

Customer app (Android, React Native)

1. **Onboarding & login** – phone number → OTP (mock in Phase 1) → name, email (optional), default address, dietary note.
2. **Home** – today's Lunch and Dinner cards with menu items, protein/kcal badge, status of the customer's meal (Scheduled / Skipped / Out for delivery / Delivered), countdown to cutoff, quick "Skip today" button.
3. **Menu** – week view (Mon–Sat) with tabs for Lunch/Dinner; tap a day for item details and photo. Shows date overrides.
4. **Plans & checkout** – list plans per slot with price; choose start date, address, flavour preference; review; pay (Phase 1: UPI details + enter UTR/upload screenshot).
5. **Book a meal (trial/daily)** – pick date & slot, see menu, add extras, choose address, pay.
6. **My Plan** – calendar of scheduled/skipped/delivered days, meals remaining, skips used/allowed, end date. Tap a day: skip/unskip, add extras, change flavour or address for that day (before cutoff). Bulk skip / pause.
7. **Add-ons** – browse extras and attach to a date.
8. **Orders & payments** – history of plans, bookings, add-ons, payment status, receipts.
9. **Profile** – addresses (home/office, with landmark and map pin), default flavour, notification settings, logout, delete account.
10. **Notifications** – in-app list: payment confirmed, plan expiring (2 meals left), cutoff reminder, admin broadcasts.
11. **Support** – "Chat on WhatsApp" deep link to the kitchen number.

Admin panel (web, responsive so it works on a phone)

1. **Dashboard** – today's counts per slot (subscribed, booked, skipped, add-ons), pending payments, plans expiring this week.
2. **Kitchen prep sheet** – for a date and slot: total meals, count per base item, count per chicken flavour, add-on counts. Printable.
3. **Delivery list** – for a date and slot: customer, phone, address, items, add-ons, notes; filter by area; assign delivery person; mark delivered/missed; export CSV.
4. **Menu management** – edit weekday template per slot (items, description, photo, protein g, kcal); create date overrides; preview the week.
5. **Add-ons** – CRUD, price, slot, stock, active toggle.
6. **Plans & pricing** – CRUD plan types: slot, meal count, price, skip limit, validity, trial-once flag, active.
7. **Customers** – search, profile, active plans, delivery calendar; admin actions: skip/unskip on behalf, grant credits, extend plan, cancel plan, record refund, notes.
8. **Payments** – queue of pending payments with UTR/screenshot; approve/reject; record cash; refund log.

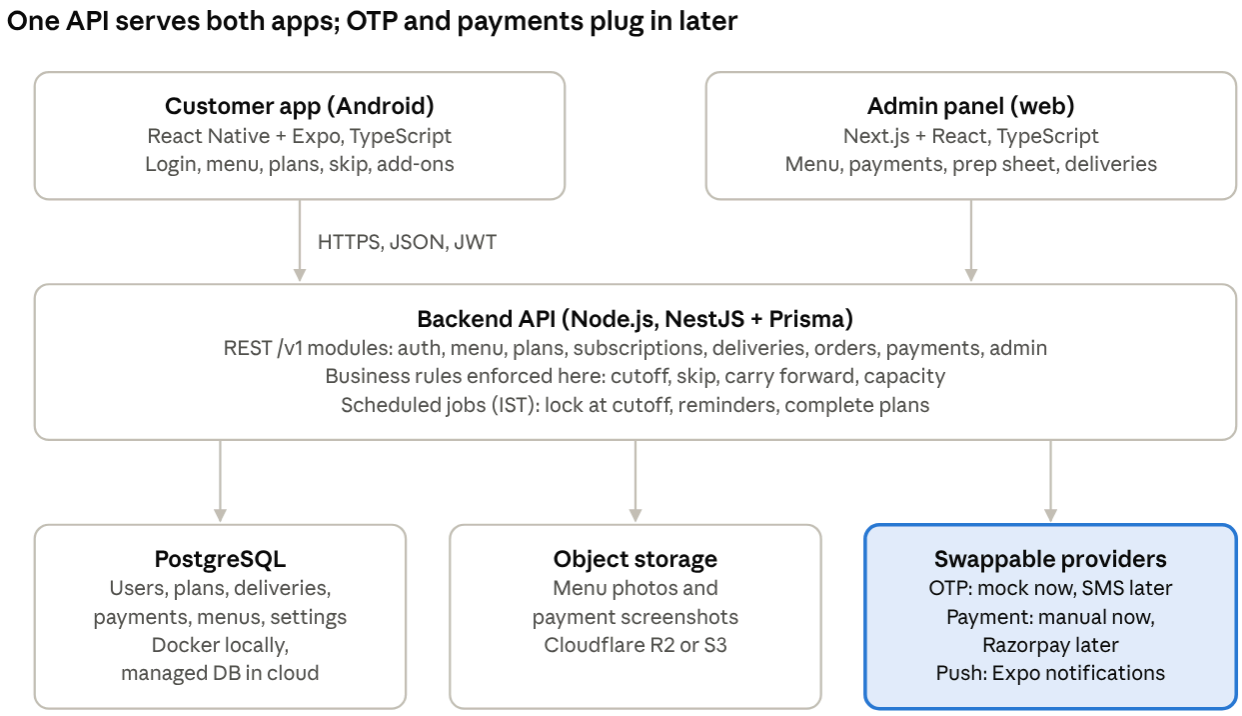
9. **Settings** – cutoff times per slot, service weekdays, holidays calendar, max booking days ahead, daily capacity per slot, UPI ID/QR, WhatsApp number, delivery areas/pincodes.
10. **Staff & broadcast** – manage admin/kitchen/delivery users and roles; send an in-app notice to all or to active subscribers.

Non-functional

- Works on mid-range Android (Android 8+), Malayalam-ready i18n structure (English first).
- Cutoff and skip rules enforced on the server, never only in the app.
- Audit log of every admin change to menus, prices, credits and payments.
- Data import: CSV upload of existing WhatsApp subscribers (name, phone, address, plan, meals remaining).

Architecture and tech stack

One TypeScript monorepo: an Expo app for customers, a Next.js admin panel and a NestJS API on PostgreSQL. All business rules live in the API; OTP and payments sit behind interfaces so Phase 2 swaps implementations, not code paths.



system architecture · 2 clients, 1 API, 3 backing services

Both clients call the same REST API; the highlighted providers are the only parts that change in Phase 2.

Layer	Choice	Why
Customer app	React Native + Expo (TypeScript), Expo Router, TanStack Query, Zustand, EAS Build	One codebase for Android now and iOS later; APK builds without Play Store
Admin panel	Next.js (App Router) + Tailwind + shadcn/ui	Fast to build tables and forms; works on the owner's phone browser
API	Node.js 20, NestJS, Prisma, Zod, Swagger/OpenAPI	Clear module structure that a coding agent follows well
Database	PostgreSQL 16	Transactions for skip/carry-forward; relational reports
Jobs	@nestjs/schedule (single API instance)	Cutoff locks and reminders without extra infra
Files	S3-compatible storage (Cloudflare R2/S3; MinIO locally)	Menu photos, payment screenshots
Hosting	API + DB on Railway or Render; admin on Vercel	Low-cost tiers suit a single kitchen; move later if needed
Tests	Jest + Supertest; Testcontainers Postgres	Rule tests run against a real database
Shared	pnpm workspaces + Turborepo, `packages/shared`	Shared types, Zod schemas, date/cutoff helpers

Repo layout

```
way2meal/
  apps/
    mobile/        # Expo customer app
    admin/         # Next.js admin panel
    api/           # NestJS API
(src/modules/{auth,menu,plans,subscriptions,deliveries,orders,payments,admin,
notifications})
  packages/
    shared/        # types, zod schemas, IST date + cutoff utils
  docs/REQUIREMENTS.md
  docker-compose.yml   # postgres, minio
  CLAUDE.md
```

Provider interfaces (implement mock/manual now, real ones in Phase 2)

```
interface OtpProvider { send(phone: string, code: string): Promise<void>; }
// MockOtpProvider: logs code, accepts DEV_OTP=123456 in non-prod; admin
panel shows latest OTPs

interface PaymentProvider {
  createIntent(p: { amountPaise: number; refType: 'SUBSCRIPTION'|'ORDER';
refId: string }): Promise<{ providerRef?: string; instructions: object }>;
  verify(paymentId: string, data: unknown):
Promise<'PAID'|'FAILED'|'PENDING'>;
  refund(paymentId: string, amountPaise: number): Promise<void>;
}
// ManualPaymentProvider: returns UPI ID/QR; verify() is the admin approval
```

Security: OTP codes hashed, rate limits on auth, role guards on every `/admin` route, signed upload URLs, secrets in environment variables only.

Data model

The central table is `Delivery`: one row per meal per date, for both subscriptions and one-off bookings, so the kitchen sheet, delivery list and skip logic all read one place. Sketch in Prisma syntax (agent may refine):

```
enum Role { CUSTOMER ADMIN KITCHEN DELIVERY }
enum Slot { LUNCH DINNER }
enum PlanKind { TRIAL DAILY WEEKLY MONTHLY }
enum SubStatus { PENDING_PAYMENT ACTIVE PAUSED COMPLETED CANCELLED }
enum DeliveryStatus { SCHEDULED SKIPPED PREPARING OUT_FOR_DELIVERY DELIVERED
MISSED CANCELLED }
enum PaymentStatus { PENDING SUBMITTED PAID REJECTED REFUNDED }

model User { id, phone @unique, name, email?, role Role, defaultFlavourId?,
createdAt, addresses Address[] }
model Address { id, userId, label (Home/Office), line1, line2?, landmark?,
area, pincode, lat?, lng?, isDefault }
model OtpRequest { id, phone, codeHash, expiresAt, attempts, consumedAt? }

model MenuItem { id, name, description?, imageUrl?, proteinG?, kcalMin?,
kcalMax?, active }
model MenuTemplate { id, weekday (1-7), slot Slot, items MenuTemplateItem[] }
// @@unique([weekday, slot])
model MenuOverride { id, date, slot Slot, items MenuOverrideItem[], note? }
// @@unique([date, slot])
model Flavour { id, name, imageUrl?, active, sortOrder }

model AddOn { id, name, description?, imageUrl?, pricePaise, slots Slot[],
```

```
dailyStock?, active }

model Plan { id, name, kind PlanKind, slot Slot, mealCount, pricePaise,
maxSkips, validityDays, oncePerUser, active }
model Subscription { id, userId, planId, slot, status SubStatus, startDate,
endDate, mealsTotal, skipsUsed, extraCredits, addressId, flavourId?,
pricePaise, createdAt }

model Delivery {
  id, userId, slot, date, status DeliveryStatus,
  subscriptionId?, orderId?,             // exactly one is set
  addressId, flavourId?, note?,
  isCarryForward Boolean, carriedFromId?, // links appended row to the
skipped one
  assignedToId?, deliveredAt?, updatedAt
  // @@index([date, slot, status])
}
model DeliveryAddOn { id, deliveryId, addOnId, qty, pricePaise, paymentId?,
status }

model Order { id, userId, kind (TRIAL|DAILY|ADDON), totalPaise, status,
createdAt }   // one-off bookings and add-on purchases
model Payment { id, userId, subscriptionId?, orderId?, amountPaise, method
(UPI_MANUAL|CASH|RAZORPAY), status PaymentStatus, utr?, screenshotUrl?,
providerRef?, verifiedById?, verifiedAt?, notes? }
model Refund { id, paymentId, amountPaise, reason, recordedById, createdAt }

model Holiday { id, date, slot? (null = both), reason }
model Setting { key @id, value Json }   // cutoffs, serviceWeekdays,
bookingDaysAhead, capacity, upiId, whatsapp, areas
model Notification { id, userId?, title, body, readAt?, createdAt }
model AuditLog { id, actorId, action, entity, entityId, before Json?, after
Json?, createdAt }
```

Key invariants (enforce in service layer + tests)

- Per subscription, count of deliveries in (SCHEDULED, PREPARING, OUT_FOR_DELIVERY, DELIVERED, MISSED) = `mealsTotal + extraCredits`.
- At most one non-cancelled delivery per user + slot + date per subscription.
- No customer write to a delivery whose date/slot is past cutoff.
- Skip, unskip and carry-forward run inside one DB transaction.

API endpoints

REST over HTTPS, JSON, JWT access token (15 min) + refresh token (30 days); `/v1` prefix; OpenAPI spec generated from code.

Area	Method & path	Who	Purpose
Auth	POST /auth/otp/request	Public	Send OTP via `OtpProvider` (rate-limited 3/10 min per phone)
Auth	POST /auth/otp/verify	Public	Verify, create user if new, return tokens
Auth	POST /auth/refresh, POST /auth/logout	Customer	Token refresh / revoke
Profile	GET/PATCH /me, CRUD /me/addresses	Customer	Profile and addresses
Menu	GET /menu?from=&to=&slot=	Public	Resolved menu per date (override else template), holidays flagged
Catalogue	GET /plans, GET /addons, GET /flavours	Public	Active items with prices
Subscriptions	POST /subscriptions	Customer	Create plan purchase → PENDING_PAYMENT
Subscriptions	GET /subscriptions, GET /subscriptions/:id	Customer	Plans with remaining meals, skips, end date
Deliveries	GET /deliveries?from=&to=	Customer	Calendar of own deliveries
Deliveries	POST /deliveries/:id/skip, POST /deliveries/:id/unskip	Customer	Skip/unskip with carry forward (cutoff-checked)
Deliveries	POST /subscriptions/:id/skip-range	Customer	Bulk skip / pause
Deliveries	PATCH /deliveries/:id	Customer	Change address, flavour, note before cutoff

Area	Method & path	Who	Purpose
Deliveries	POST/DELETE /deliveries/:id/addons	Customer	Attach or remove extras
Orders	POST /orders	Customer	Trial/daily booking (+ add-ons), capacity-checked
Orders	POST /orders/:id/cancel	Customer	Cancel before cutoff
Payments	POST /payments/:id/submit	Customer	Submit UTR + screenshot upload
Payments	GET /payments	Customer	History
Notifications	GET /notifications, POST /notifications/:id/read	Customer	In-app inbox
Admin	GET /admin/dashboard?date=	Admin	Counts and alerts
Admin	GET /admin/prep-sheet?date=&slot=	Admin, Kitchen	Item, flavour and add-on counts
Admin	GET /admin/deliveries? date=&slot=&area=, PATCH /admin/deliveries/:id	Admin, Kitchen, Delivery	List, assign, status, export CSV
Admin	CRUD /admin/menu-items, /admin/menu-templates, /admin/menu-overrides, /admin/flavours	Admin	Menu management
Admin	CRUD /admin/addons, /admin/plans	Admin	Catalogue and pricing
Admin	GET /admin/customers, GET /admin/customers/:id, POST /admin/subscriptions/:id/{grant-credit,extend,cancel}	Admin	Customer actions
Admin	GET /admin/payments?status=, POST /admin/payments/:id/{approve,reject}, POST /admin/payments/cash, POST /admin/refunds	Admin	Payment verification
Admin	CRUD /admin/holidays, GET/PUT /admin/settings, CRUD /admin/staff	Admin	Configuration

Area	Method & path	Who	Purpose
Admin	POST /admin/broadcast, POST /admin/import/customers	Admin	Notices and CSV import
Webhooks	POST /webhooks/razorpay	Razorpay	Phase 2 only; signature-verified

Scheduled jobs (daily, IST): at each slot's cutoff lock that slot's deliveries to PREPARING; at 8 PM send "2 meals left" and next-day cutoff reminders; at midnight mark finished plans COMPLETED.

Seed data from current menus

Load this as the initial weekday templates; every meal is tagged 35 g+ protein, 350–450 kcal, "Balanced meal".

Day	Lunch	Dinner
Monday	Smart Ghee Rice, Herbal Chicken Breast, Veg Salad, Yogurt	Wheat Macaroni, 100 g Chicken Breast (cooked), Fruits
Tuesday	Smart Mandi, Steamed Chicken Breast, Veg Salad, Yogurt	Oats Upma, 100 g Chicken Breast (cooked), Fruits
Wednesday	Tomato Rice, Steamed Chicken Breast, Veg Salad, Yogurt	Wheat Pasta, 100 g Chicken Breast (cooked), Fruits
Thursday	Smart Biriyani, Steamed Chicken Breast, Veg Salad, Yogurt	Wheat Upma, 100 g Chicken Breast (cooked), Fruits
Friday	Smart Fried Rice, Steamed Chicken Breast, Veg Salad, Yogurt	Wheat Noodles, 100 g Chicken Breast (cooked), Fruits
Saturday	Brown Rice, Chicken Breast Curry, Thoran, Curd	Wheat Macaroni, 100 g Chicken Breast (cooked), Fruits

- **Veg Salad** = broccoli, lettuce, red cabbage, cucumber, carrot.
- **Flavours** (dinner chicken, 100 g cooked): Flavour 1–6 — placeholder names until the kitchen supplies real ones.
- **Add-ons:** Premium Protein Veg Salad 300 g (100 g chicken breast + 200 g veg salad) – price TBD; Extra Chicken Breast 100 g – price TBD; Extra Fruit Bowl – price TBD; Extra Yogurt/Curd – price TBD.

- **Plans:** Lunch Trial ₹169, Lunch Weekly (6) ₹990, Lunch Monthly (26) ₹4159; Dinner Trial ₹169, Dinner Weekly (6) ₹960, Dinner Monthly (26) ₹4160.
- **Settings:** cutoff 10:00 AM both slots; service days Mon–Sat; refund and carry-forward enabled.

Open questions for the kitchen

- [ ] Prices for each add-on and for a single "daily" meal (if different from ₹169)
- [ ] Real names of the 6 chicken flavours
- [ ] Is Dinner cutoff also 10 AM, or later?
- [ ] Max skips per weekly/monthly plan, and how long carried-forward meals stay valid
- [ ] Delivery areas/pincodes and daily capacity per slot
- [ ] Combo price if someone takes both Lunch and Dinner plans

Milestones, acceptance criteria and Claude Code instructions

Build in five milestones, each ending in something the kitchen can try; hand Claude Code one milestone at a time.

1. **M1 – Foundation:** monorepo, Docker Postgres, Prisma schema + migrations, seed data, auth with mock OTP, CI running lint + tests.
2. **M2 – Menu & catalogue:** admin CRUD for menu templates, overrides, flavours, add-ons, plans, settings, holidays; customer app Home + Menu screens.
3. **M3 – Subscriptions & skip:** plan purchase, manual payment submit/approve, delivery scheduling, skip/unskip/carry-forward, My Plan calendar.
4. **M4 – Bookings & add-ons:** trial/daily orders with capacity and cutoff, add-ons on deliveries and orders.
5. **M5 – Kitchen ops:** dashboard, prep sheet, delivery list + CSV, customer admin actions, CSV import, notifications, scheduled jobs; signed APK build.
6. **Phase 2:** `RazorpayPaymentProvider` (orders + webhooks + refunds), real `OtpProvider` (MSG91/Twilio with DLT-registered template), Play Store listing, WhatsApp notifications.

Acceptance criteria (must have automated tests)

- [ ] Monthly plan bought Monday with start Tue creates 26 deliveries on Mon–Sat only, skipping holidays.
- [ ] Skipping a day before 10:00 IST marks it SKIPPED, appends one delivery after the last one, and moves `endDate`.
- [ ] Skipping at 10:01 IST on the same day is rejected with a clear error.
- [ ] Unskip restores the date and removes the appended row; meal count stays constant.

- [ ] Skip limit reached → API returns 409 with the limit in the message.
- [ ] Holiday added after purchase moves affected deliveries to the end of the plan.
- [ ] Trial can be bought once per phone; booking fails when slot capacity is full.
- [ ] Admin menu override for a date shows in the app instead of the template.
- [ ] Prep sheet totals equal the count of non-skipped deliveries + bookings for that date/slot, with flavour and add-on breakdowns.
- [ ] Plan stays PENDING_PAYMENT until admin approves; approval schedules deliveries.

Prompt to give Claude Code (paste with this doc exported as Markdown, saved as `docs/REQUIREMENTS.md`):

```
You are building Phase 1 of the Way2meal diet-food delivery app. Read
docs/REQUIREMENTS.md fully before writing code.

1. Create CLAUDE.md summarising the stack, folder layout, commands, and the
business rules in the "Business rules" section.
2. Work milestone by milestone (M1 first). At the start of each milestone,
write a short plan; at the end, run lint, type-check and tests and show me
the results.
3. Put all payment and OTP code behind the PaymentProvider and OtpProvider
interfaces. Implement only ManualPaymentProvider and MockOtpProvider now.
4. Enforce cutoff, skip limits and carry-forward on the server in a
DeliveryService with unit tests for every acceptance criterion. Use an
injectable clock so tests can set IST times.
5. Store money in paise (integers), times in UTC, and compute service days in
Asia/Kolkata.
6. Never hard-code menu, prices or limits; read them from the
database/settings.
7. Ask me before adding any paid third-party service.

Start with M1.
```